Министерство образования Республики Беларусь

Учреждение образования

«Брестский Государственный технический университет»

Кафедра ИИТ

Лабораторная работа №2

По дисциплине «Современные методы защиты компьютерных систем»

Тема: «Защита веб-периметра и ограничение интенсивности сетевого трафика средствами межсетевых экранов прикладного уровня»

Выполнила:

Студент 4 курса

Группы ИИ-26

Карпович К.А.

Проверила:

Хацкевич А.С.

Брест 2026

Цель работы: практическое освоение методов развертывания межсетевых экранов прикладного уровня (WAF), написания пользовательских правил фильтрации L7-угроз, а также проектирование механизмов защиты от DDoS-атак и ограничения частоты запросов (Rate Limiting) на базе обратного прокси-сервера Nginx.

Инструкция по подготовке и запуску стенда:

1. Установите Docker и Docker Compose (рекомендуемая среда для развертывания Nginx + WAF).
2. Для задания 3 вам потребуется утилита Apache Bench (ab). На Linux/WSL она устанавливается командой sudo apt install apache2-utils. На Windows можно использовать аналоги — утилиты hey или bombardier.
3. Перейдите в директорию стенда (где находится файл docker-compose.yml) и выполните команду: docker-compose up -d.
4. Убедитесь, что уязвимое веб-приложение доступно напрямую по порту 5000 (http://127.0.0.1:5000), а защищенный контур через Nginx WAF доступен по порту 80 (http://127.0.0.1).
5. Для выполнения всех шагов с вашего рабочего компьютера (терминала) используются консольные утилиты curl и ab.

Вариант	WAF: Блокируемый User-Agent	ID правила WAF	Rate Limit: Эндпоинт (Target)	Rate Limit: rate	Rate Limit: burst
7	`ZGrab/0.x`	10007	`/search`	`5r/s`	`15`

Стенд был развернут с использованием Docker Compose (команда docker-compose up -d). Уязвимое веб-приложение стало доступно напрямую по адресу http://127.0.0.1:5000, а защищенный контур через Nginx WAF – по адресу http://127.0.0.1.

Задание 1. Базовая фильтрация угроз через WAF (OWASP Core Rule Set):

1.Проведение SQLi-атаки напрямую к бэкенд-приложению (в обход WAF)

Для проверки уязвимости приложения был отправлен вредоносный SQL-запрос напрямую на порт 5000 с помощью утилиты curl: `curl.exe -i "http://127.0.0.1:5000/?id=1'%20OR%20'1'='1"`

Результат: Сервер вернул код HTTP/1.1 200 OK, что подтверждает отсутствие защиты на уровне бэкенда и успешное выполнение инъекции.

2. Проведение SQLi-атаки через защищенный Nginx-периметр

Аналогичный SQLi-запрос был отправлен через обратный прокси-сервер Nginx с активированным WAF (порт 80): `curl.exe -i "http://127.0.0.1/?id=1'%20OR%20'1'='1"`

3. Фиксация результатов блокировки атаки базовыми правилами WAF

```
PS C:\Users\Ксения\Downloads\SMZKS-2026-autumn-main\labs\lab2>    curl.exe -i "http://127.0.0.1:5000/?id=1'%20OR%20'1'='
1"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.9 Python/3.10.22
Date: Tue, 13 Oct 2026 21:42:28 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 31
Connection: close

Backend OK. Endpoint reached: /
PS C:\Users\Ксения\Downloads\SMZKS-2026-autumn-main\labs\lab2>    curl.exe -i "http://127.0.0.1/?id=1'%20OR%20'1'='1"
HTTP/1.1 403 Forbidden
Server: nginx/1.30.5
Date: Tue, 13 Oct 2026 21:45:57 GMT
Content-Type: text/html
Content-Length: 153
Connection: keep-alive

<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.30.5</center>
</body>
</html>
```

Результат: WAF на базе ModSecurity и OWASP CRS перехватил атаку, вернув код HTTP/1.1 403 Forbidden.

Задание 2. Написание пользовательских правил фильтрации (L7):

Согласно варианту №7, необходимо блокировать сканер уязвимостей с заголовком User-Agent: ZGrab/0.x. Уникальный идентификатор правила: 10007.

1. Разработка и внедрение пользовательского правила (SecRule) в файл custom.conf

В конфигурационный файл nginx/custom.conf было добавлено следующее правило на языке SecRules:
`SecRule REQUEST_HEADERS:User-Agent "@contains ZGrab/0.x" "id:10007,phase:1,deny,status:403,msg:'Blocked ZGrab scanner'"`

После внесения изменений контейнер WAF был перезапущен командой `docker-compose restart waf`.

2. Верификация блокировки запроса с подмененным заголовком User-Agent

Для проверки было отправлено запрос с подменой заголовка: `curl.exe -i -H "User-Agent: ZGrab/0.x" http://127.0.0.1/`

```
PS C:\Users\Ксения\Downloads\SMZKS-2026-autumn-main\labs\lab2>    docker-compose restart waf
WARN[0000] C:\Users\Ксения\Downloads\SMZKS-2026-autumn-main\labs\lab2\docker-compose.yml: the attribute `version` is obs
olete, it will be ignored, please remove it to avoid potential confusion
[+] restart 0/1
 - Container lab2-waf-1 Restarting                                                                                  1.4s
PS C:\Users\Ксения\Downloads\SMZKS-2026-autumn-main\labs\lab2>    curl.exe -i -H "User-Agent: ZGrab/0.x" http://127.0.0.
1/
HTTP/1.1 403 Forbidden
Server: nginx/1.30.5
Date: Tue, 13 Oct 2026 21:54:31 GMT
Content-Type: text/html
Content-Length: 153
Connection: keep-alive

<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.30.5</center>
</body>
</html>
PS C:\Users\Ксения\Downloads\SMZKS-2026-autumn-main\labs\lab2>
```

Задание 3. Защита от L7 DDoS-атак (Rate Limiting):

Согласно варианту №7, настройка производится для эндпоинта /search со следующими параметрами:

- rate: 5r/s (5 запросов в секунду)
- burst: 15 (размер очереди для всплесков трафика)

1. Конфигурирование директив ограничения интенсивности трафика в user_nginx.conf

В файл nginx/user_nginx.conf были добавлены директивы `limit_req_zone` и `limit_req`:

```
limit_req_zone $binary_remote_addr zone=mylimit:10m rate=5r/s;

server {

    listen 80;

    server_name localhost;

    location /search {

        limit_req zone=mylimit burst=15 nodelay;

        limit_req_status 503;


        proxy_pass ${BACKEND};

        proxy_set_header Host $host;

        proxy_set_header X-Real-IP $remote_addr;

        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;

    }
```

```
    location / {
        proxy_pass ${BACKEND};
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

2. Проведение стресс-тестирования и анализ ответов сервера (код 503)

В соответствии с методическими указаниями для ОС Windows, вместо утилиты Apache Bench (ab) был использован встроенный скрипт PowerShell, эмулирующий отправку 500 HTTP-запросов на эндпоинт /search (аналог команды ab -n 500 -c 10 http://127.0.0.1/search).

Результаты стресс-теста:

- Успешных запросов (200 OK): 27
- Заблокированных запросов (503 Service Unavailable): 473

```
PS C:\Users\Ксения\Downloads\SMZKS-2026-autumn-main\labs\lab2>    docker-compose restart waf
WARN[0000] C:\Users\Ксения\Downloads\SMZKS-2026-autumn-main\labs\lab2\docker-compose.yml: the attribute `version` is obs
olete, it will be ignored, please remove it to avoid potential confusion
[+] restart 0/1
 - Container lab2-waf-1 Restarting                                                                                  1.2s
PS C:\Users\Ксения\Downloads\SMZKS-2026-autumn-main\labs\lab2>    $success = 0
PS C:\Users\Ксения\Downloads\SMZKS-2026-autumn-main\labs\lab2>    $failed = 0
PS C:\Users\Ксения\Downloads\SMZKS-2026-autumn-main\labs\lab2>    for ($i = 1; $i -le 500; $i++) {
>>        try {
>>            $response = Invoke-WebRequest -Uri "http://127.0.0.1/search" -UseBasicParsing -TimeoutSec 5
>>            if ($response.StatusCode -eq 200) { $success++ } else { $failed++ }
>>        } catch {
>>            $failed++
>>        }
>>    }
PS C:\Users\Ксения\Downloads\SMZKS-2026-autumn-main\labs\lab2>    Write-Host "Успешных запросов (200 OK): $success"
Успешных запросов (200 OK): 27
PS C:\Users\Ксения\Downloads\SMZKS-2026-autumn-main\labs\lab2>    Write-Host "Заблокированных запросов (503 или ошибки):
 $failed"
Заблокированных запросов (503 или ошибки): 473
PS C:\Users\Ксения\Downloads\SMZKS-2026-autumn-main\labs\lab2>
```

Вывод: Механизм Rate Limiting корректно обработал разрешенный всплеск (burst=15), а все избыточные запросы сверх лимита (5r/s) были отброшены с кодом 503, что подтверждает защиту бэкенда от L7 DDoS-атак.

Задание 4 (Проектирование профилей защиты):

1. Настройка параметров Nginx для защиты от атаки «Slowloris» с техническим обоснованием

Для противодействия атаке Slowloris (медленное исчерпание пула соединений) предлагается следующая конфигурация таймаутов и лимитов:

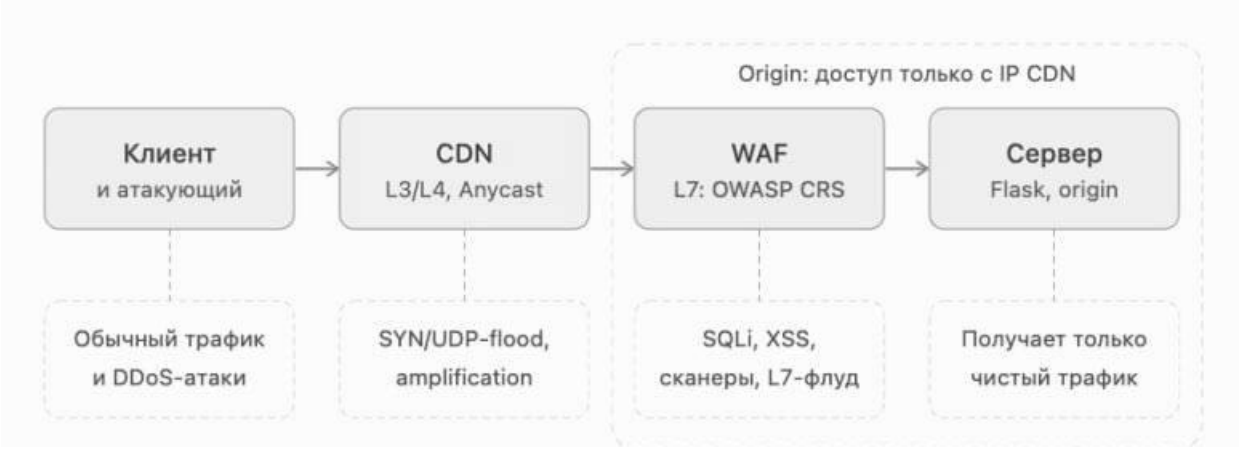
```
        client_header_timeout 10s;
        client_body_timeout   10s;
        keepalive_timeout     15s;
        send_timeout          10s;
        reset_timedout_connection on;
        limit_conn_zone $binary_remote_addr zone=perip:10m;
        limit_conn perip 20;
```

Обоснование:

- Атака держит соединения открытыми, медленно досылая заголовки или тело. Значение по умолчанию (60 с) слишком щедрое: 10 с достаточно для заголовков легитимного клиента даже при плохой связи, а медленные соединения быстро освобождают worker connections.
- client_body_timeout 10s задаёт паузу между чтениями тела, а не общее время загрузки, поэтому обычные загрузки не ломаются. Для эндпоинтов с большими загрузками значение можно поднять в отдельном location.
- keepalive_timeout 15s не даёт простаивающим соединениям занимать слоты.
- limit_conn ограничивает число одновременных соединений с одного IP, а reset_timedout_connection on сразу освобождает память сокета при таймауте.
- Слишком малые значения (1–2 с) приведут к ложным разрывам у пользователей с медленным каналом, поэтому выбран баланс 10–15 с.

2. Проектирование схемы эшелонированной защиты (CDN + WAF) от объемных DDoS-атак

Архитектурная схема: Пользователь → CDN (L3/L4 защита) → WAF (L7 защита) → Сервер приложения (Origin)



Принцип эшелонированной защиты:

- Объёмные L3/L4-атаки (десятки и сотни Гбит/с) гасит CDN: его канал и распределённая инфраструктура больше, чем у твоего сервера. Кэш статики дополнительно разгружает origin.
- WAF отсекает то, что CDN не видит по смыслу: SQLi, XSS, сканеры, перебор и L7-флуд.
- Origin должен быть скрыт: его IP не публикуется, файрвол пропускает на 80/443 только диапазоны CDN (при желании добавляется mTLS между CDN и WAF).
- Чтобы rate limiting на WAF работал по реальным клиентам, а не по IP CDN, настраивается set_real_ip_from с диапазонами CDN и real_ip_header X-Forwarded-For.

Вывод: в ходе работы был развёрнут стенд Nginx + ModSecurity с набором правил OWASP CRS: WAF заблокировал SQL-инъекцию, тогда как напрямую бэкенд принимал её, а кастомное правило SecRule остановило сканер ZGrab/0.x. Rate limiting (5r/s, burst=15) на /search показал, что избыточные запросы L7-флуда отбрасываются с кодом 503 и бэкенд защищён от перегрузки, а проработанные профили против Slowloris и схема CDN + WAF закрывают медленные и объёмные атаки. Эшелонированная защита из нескольких механизмов оказалась эффективнее любого одиночного средства.